

EuroScope

Plug-in developer's Guide

For version 3.1d (r4)

27.06.2010

Table of contents

The plug-in development	4
Plug-in development environment	4
What is the development environment?	4
Quick look to the structure	5
EuroScopePlugIn namespace	5
Constants definitions	5
CPosition class	5
CAircraftExtractedRoute class	5
CAircraftPositionPredictions class	6
CAircraftPositionData class	6
CAircraftFlightPlan class	6
CAircraftControllerAssignedData class	6
CAircraft class	6
CController class	6
CRadarScreen class	6
CPlugIn class	7
Global exported functions	7
Example plug-in step-by-step	7
New DLL project	7
An empty CPlugIn sub class	10
Adding the public exported functions	11
My own radar screen	12
OnAsrContentLoaded	13
OnAsrContentToBeSaved	13
OnRefresh	13
OnCompileCommand	13
Register and create the radar screen	13
Debugging a plug-in	14

The plug-in development

The plug-in development offers the possibility to add your own code to EuroScope. In a plug-in you will be able to:

- be informed about all the actions in the session (even in a playback one),
- draw anything into the radar screen,
- define your own unique radar display,
- store and recall additional data in the ASR files,
- add new TAG items (not yet implemented),
- manipulate data available for the controller (not yet implemented).

Plug-in development environment

EuroScope is programmed in C++ using MFC (Microsoft Foundation Classes). The actual version is created by Microsoft Visual Studio 2005 using MFC and CRT 8.0.50727.762. All the necessary DLLs are included in the installer. For plug-in development I do suggest using the very same versions as in this case you only need to publish your final DLL and rest is available in the EuroScope environmet.

But this is not restricted at all. You may use different CRT and MFC. But in this case you have to publish all necessary files that are required by your DLL to be loaded successfully.

All the interfaces in the plug-in development environment are built on C++ and some Windows GDI, but never used any MFC. It is up to you if you are using MFC in the plug-in or not. I use MFC just for causal programming.

All classes are designed to store only on or two pointers, but no more member variables. This allows me to change the underlying structure hidden from the plug-ins and still possible to load the old plug-ins.

What is the development environment?

The development environment is actually no more than two files:

- **EuroScopePlugIn.h** – This file contains all the class definitions you need for the plug-in. Just include it into your sources and implement some functions and subclasses.
- **EuroScopePlugInDll.lib** – This is auto loading DLL library. It contains all the exported features from **EuroScope.exe**. Just link this library to your DLL.

Important changes

Changes from v3.1 to 3.1d(r4)

There was a huge restructuring of the classes to 3.1d (r4). I cancelled the use of CAircraft class and introduced the CFlightPlan and CRadarTarget classes. It is now in complete synergy with the classes in the main code. The functions of CAircraft are split between the new classes. There are also some pure naming change to avoid using the name Aircraft. In this case just use a search and replace along your code:

- AIRCRAFT_STATE... is renamed to FLIGHT_PLAN_STATE...
- CAircraftExtractedRoute is renamed to CFlightPlanExtractedRoute
- CAircraftPositionPredictions is renamed to CFlightPlanPositionPredictions
- CAircraftPositionData is renamed to CRadarTargetPositionData
- CAircraftFlightPlan is renamed to CFlightPlanData
- CAircraftControllerAssignedData is renamed to CFlightPlanControllerAssignedData
- CAircraft class is deleted
- CFlightPlan class is introduced
- CRadarTarget class is introduced
- OnAircraftPositionUpdate is renamed to OnRadarTargetPositionUpdate
- OnAircraftDisconnect is renamed to OnFlightPlanDisconnect
- OnAircraftFlightPlanUpdate is renamed to OnFlightPlanFlightPlanDataUpdate
- OnAircraftControllerAssignedDataUpdate is renamed to OnFlightPlanControllerAssignedDataUpdate
- OnAircraftFlightStripPushed is renamed to OnFlightPlanFlightStripPushed
- CAircraftList is renamed to CFlightPlanList
- OnGetTagItem has now both CFlightPlan and CRadarTarget parameters and at least one of them is valid

New features:

- FlightPlan changing and amending functions for CFlightPlanData class
- Get performance data from CFlightPlanData class

Changes from v3.0d(r9) to v3.0d(r10)

There is a new way of deleting the memory allocated by the plug-ins. There are two new rules:

- All plug-in must save its **CPlugIn** derived class instance pointer that is allocated at **EuroScopePlugInInit** and delete it at **EuroScopePlugInExit**.
- All plug-ins that allocate **CRadarScreen** derived instance to communicate with a radar screen, must implement the **OnAsrContentToBeClosed** and free itself (delete this) as last action. The **OnAsrContentToBeClosed** is changed that you must implement to be able to allocate an instance.

Changes from v3.0c(r4) to v3.0d(r3)

Important change that **CPlugIn::OnGetTagItem** is extended by and additional parameters: **TagData**. It can be on of **TAG_DATA_UNCORRELATED_RADAR**, **TAG_DATA_FLIGHT_PLAN_TRACK**, **TAG_DATA_CORRELATED** to indicate what data might be available.

The **CAircraftControllerAssignedData** is extended with several **Set...** functions. You can set all controller data via these functions. So far no restriction on them, but there may be implemented some later on.

The **CAircraft** has new functions to test if the radar track and the FP is correlated or not. Also there are functions to initiate/accept/refuse track handling and coordination.

Changes from v3.0c(r3) to v3.0d(r2)

As there was a lot of problem with the old compiled plug-ins I added a compatibility test. The compilation time version number must be presented by **CPlugIn** constructor. This function has been extended by an additional **CompatibilityCode** parameter. You should pass the **EuroScopePlugIn::COMPATIBILITY_CODE** variable here. It defines the header version number. When the interface is changed in a way the plug-ins are not more compatible I simply increase this number and all old compiled plug-ins will not be loaded.

Changes from v3.0c(r2) to v3.0c(r3)

New class: **CSectorElement** to access the sector file elements. You can use these elements to display or hide them on **CRadarScreen**. New callback function **OnAirportRunwayActivityChanged** to indicate runway and airport change.

Changes from v3.0c(r2) to v3.0c(r3)

Also I am trying to remain compatible between releases, it is not always possible. Especially at the very beginning of the development, when some items must be changed to be useful. I hope there will not be long list here.

To release 3 I modified two important place, and therefore you have to change your plug-in code and recompile. Without it the old DLLs will not be loaded.

First the constructor of the **CPlugIn** class is changed. It contains the 4 name parameters that was set just after creating the instance by calling the **SetPlugInData** function. I

realized that in many cases I need these data even inside the constructor. Therefore they are now the parameter of the constructor and the **SetPlugInData** function is deleted. Change your constructor in a way to call CPlugIn with the new parameters:

```
CPrecisionApproachPlugIn::CPrecisionApproachPlugIn( void )  
    : CPlugIn (EuroScopePlugIn::COMPATIBILITY_CODE,  
               "Precision Approach Radar",  
               "1.0",  
               "Gergely Csernak",  
               "Free to be distributed as source code" )
```

The second change is that from now on ES will free all data within that was allocated especially for it. Therefore the instance of your CPlugIn derived class **MUST NOT** be freed in **EuroScopePlugInExit** as was in r2. It is deleted by ES. Calling the **delete** operator there too will cause an exception and ES crash on plug-in unload (or exit).

Quick look to the structure

Let us take a quick look to the **EuroScopePlugIn.h** and to the classes and methods there. I will not describe all the methods here as the header file itself contains probably enough comments on them, but some descriptions on the classes and the relationships.

EuroScopePlugIn namespace

All the items defined in the header file are in the **EuroScopePlugIn** namespace. In your code you need the **EuroScopePlugIn::** prefix for all references or use the **using EuroScopePlugIn** command to allow direct access.

Constants definitions

Some constants are defined at the beginning of the header. They are declaring the followings:

- flight plan states,
- route extractions classifications,
- controller assigned value types and
- refresh phases,
- TAG color definitions,
- TAG item code definitions,
- TAG item function definitions,
- button definition (mouse handling),
- popup element types (popup lists),
- connection and coordination types.

They will be references later in the functions.

CPosition class

The **CPosition** class is designed to store latitude/longitude coordinates. For easy programming there are functions to calculate the distance and direction between two points. These functions are very new in EuroScope and based on the formulas I received from Juha Holopainen. So far I added to this environment only. I use a far less precise formulas inside EuroScope, but I am planning to change it. Therefore expect some differences between the values here and the values inside EuroScope.

The direction returns the magnetic heading. The value calculated by the original function is corrected by the magnetic deviation defined in the main sectorfile.

CFlightPlanExtractedRoute class

This class stores the extracted route of one flight plan. It is actually an array of structures that describes the route of the route point by point. All segments are classified by the AIRAC data if available. Sids and stars are also extracted.

You can access to this data via a **CFlightPlan** class instance.

CFlightPlanPositionPredictions class

This class contains an array of point predictions. Every point is one minute time from the previous and describes where the plane is expected (position and altitude) and what controller is online there.

You can access to this data via a **CFlightPlan** class instance.

CRadarTargetPositionData class

This class stores one position updates of the plane. The latest is of course the actual position of the aircraft while the previous ones can be used for history analysis. Only last 20 points are available.

You can access to this data via a **CRadarTarget** or the **CFlightPlan** class instance.

CFlightPlanData class

This class stores all the information about the flight plan.

You can access to this data via a **CFlightPlan** class instance.

CFlightPlanControllerAssignedData class

It is just a collection class. It allows you to access all the data that can be assigned to an aircraft by the controller.

You can access to this data via a **CFlightPlan** class instance.

CFlightPlan class

This is the main class to store information about a flight plans. You can select a flight plan or iterate all them via the **CPlugin** class instance. And you will be noticed whenever a flight plan update is received or when a controller assigned any kind of data.

This class holds all the functions from the original CAircraft class that query or change the flight plan part.

CRadarTarget class

This is the main class to store information radar targets. You can select a radar target or iterate all them via the **CPlugin** class instance. And you will be noticed whenever a radar target position is updated.

This class holds all the functions from the original CAircraft class that query or change the radar target part.

CController class

This class is used to access to the online controllers. Like the aircrafts you can select a controller or iterate all them via the **CPlugin** class instance. And you will be noticed whenever a position update is received.

CRadarScreen class

This class is designed to hold information about a radar screen. When a new screen is created all plug-ins are offered to allocate a radar screen instance.

This class has some virtual functions that might be overridden by the plug-in. These functions will be called for screen updates, data load and save.

There is also a special function that can compile command line commands. The unrecognized command lines are first sent to the radar screens then to the plug-ins.

CFlightPlanList

This class references to a flight plan list on the screen. You do not have access to the SIL, SEL and DEP lists, but you can make your own one. You can define the columns to be displayed and feed the list with aircrafts. The new example program creates a holding list.

CPlugin class

This is the main plug-in class. All plug-ins have to subclass this one and allocate an instance in the initialization call. All subsequent interaction between EuroScope and the plug-in will use this instance.

This function has many virtual functions that might be overridden by the plug-in.

Global exported functions

There are two global functions that you **MUST** export from your DLL. When EuroScope loads the DLL it looks for the **EuroScopePlugInInit** and **EuroScopePlugInExit** functions. They must be implemented and declared as exported (as you see them in the header file).

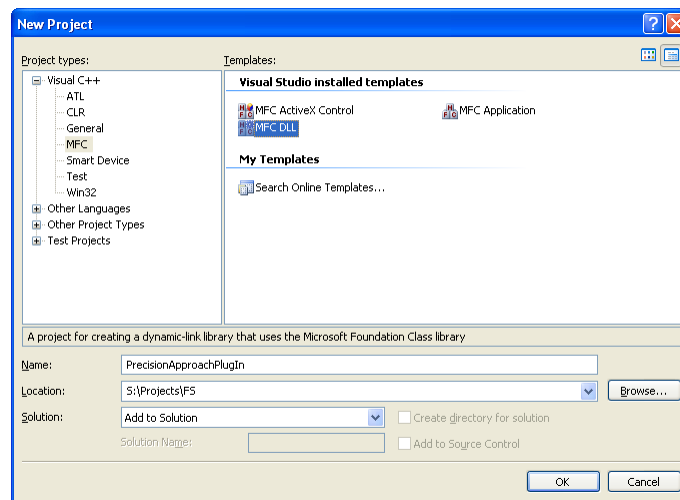
Just after the DLL loading EuroScope calls the **EuroScopePlugInInit** function. There you must allocate and return the instance of your **CPlugIn** class. Just before unload EuroScope calls the **EuroScopePlugInExit** function to clean up all local variables.

Example plug-in step-by-step

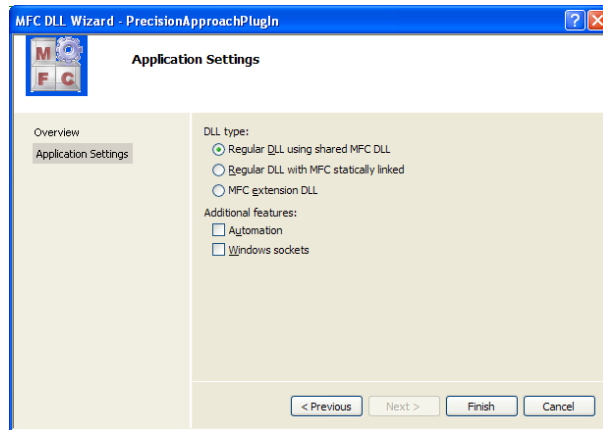
Here I describe all the steps you need to build your own plug-in. In the example I will use the “Precision Approach Plug-in” that is also available as source code for further exploration.

New DLL project

The first step is to create a new project that creates the DLL:

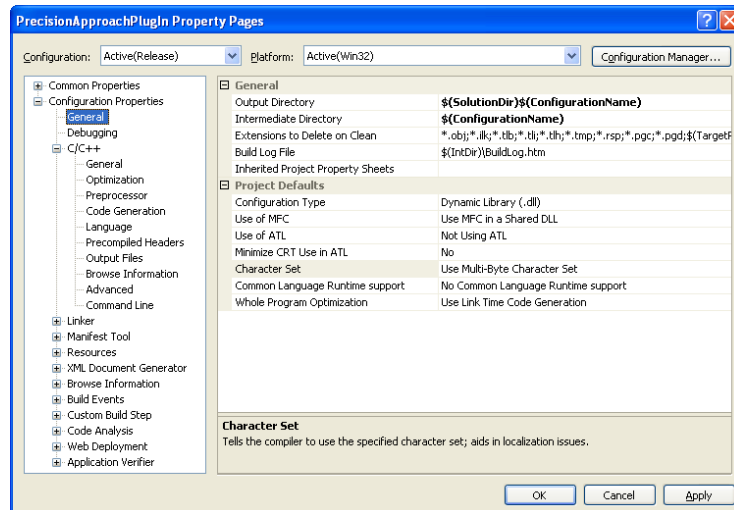


Here I started with an MFC DLL that is the convenient for me, but you can start with a DLL without MFC support.

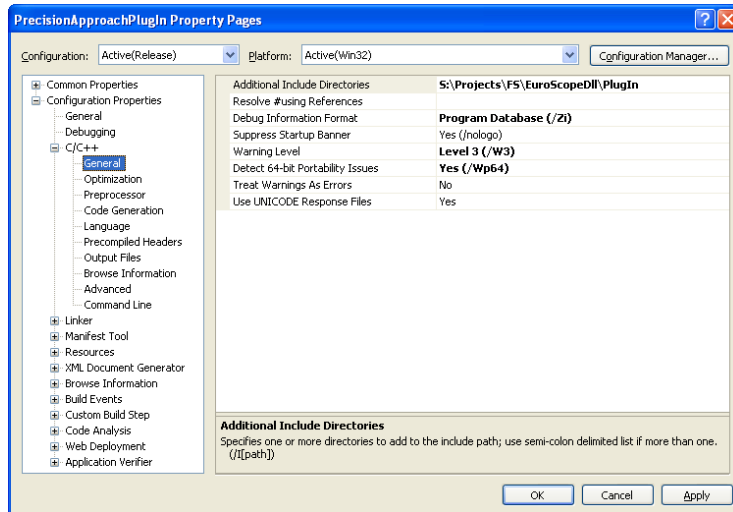


I added MFC with shared DLLs. That creates a smaller plug-in and uses the DLLs installed by EuroScope.

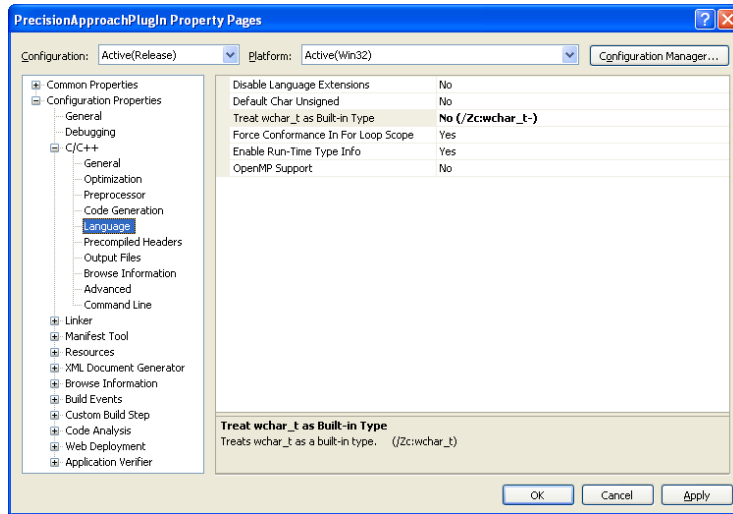
Just after the project wizard some minor adjustments have to be made in the project settings. Change the followings in both debug and release versions:



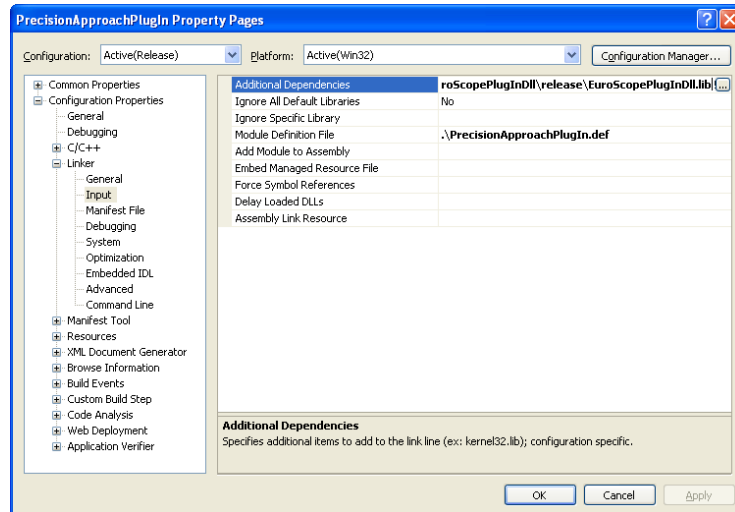
In the General tab change the *Character set* from the default Unicode to *Use Multi-Byte Character Set*. Nor the VATSIM protocol, nor EuroScope supports Unicode characters at the moment.



In the C++/General tab add the folder name where **EuroScopePlugIn.h** is located to the *Additional Include Directories*. That will enable to include it to your sources easily.



I found that disabling the *Treat wchar_t as Built-in Type* avoids some compatibility problems.



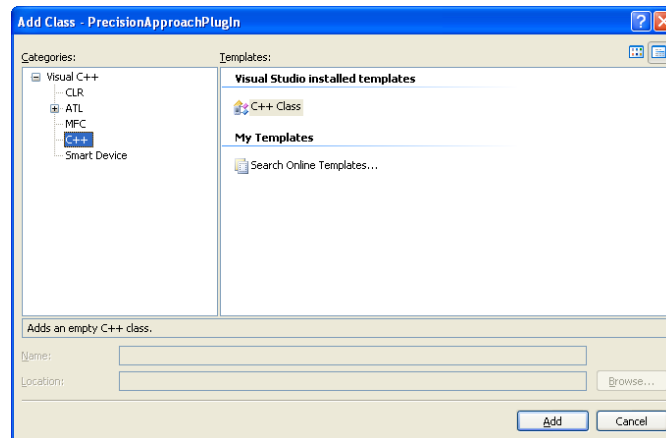
Add the **EuroScopePluginDll.lib** to the Linker/Input *Additional Dependencies*. That will link the EuroScope library to the plug-in.

To enable intelli-sense working better, I suggest adding the **EuroScopePlugin.h** to the project. Click on the *Header Files* with a right click and choose *Add/Existing Item*.

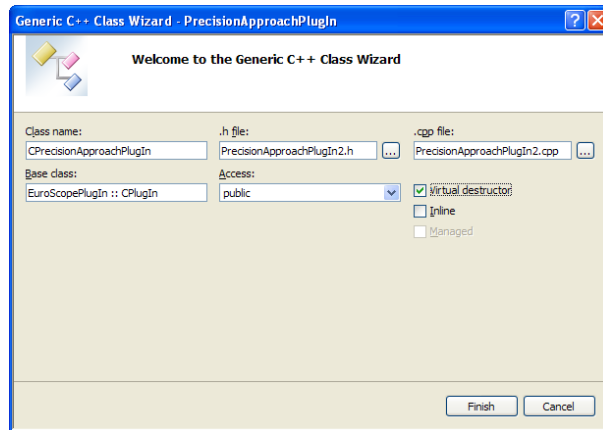
That is enough for the base project. You can start and build the empty DLL files.

An empty CPlugin sub class

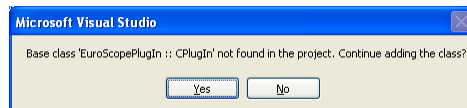
To start creating the plug-in we first need to subclass the **CPlugin** class. To do that start with the *Project/Add Class* menu:



Click *Add*.



Fill in the class name (here **CPrecisionApproachPlugIn**). I also changed the default file names adding a “2” postfix as the same file name is already used by the main code. My plug-in class has to be inherited from **EuroScopePlugIn :: CPlugIn**.



You might receive a warning message indicating that the base class is not yet found in the project. Just accept it, and continue with the class creation. We will add the necessary includes later.

Open the **PrecisionApproachPlugIn2.h** and add the include statement just after the **#pragma once** statement:

```
#include <EuroScopePlugIn.h>
```

You may compile and build the solution again.

Adding the public exported functions

Open the original **PrecisionApproachPlugIn.cpp** file. Include **PrecisionApproachPlugIn2.h** to be able to see your class:

```
#include "PrecisionApproachPlugIn2.h"
```

Copy the **EuroScopePlugInInit** and **EuroScopePlugInExit** declarations and extend it with a body. In the init just allocate a new instance while in the exit clean it up:

```
void __declspec ( dllexport )
    EuroScopePlugInInit ( EuroScopePlugIn :: CPlugIn **
ppPlugInInstance )
{
    // allocate
    * ppPlugInInstance = pMyPlugIn =
        new CPrecisionApproachPlugIn ;
}
```

```
void __declspec ( dllexport )
```

```

        EuroScopePlugInExit ( void )
    {
        delete pMyPlugIn ;
    }

```

As there is no default constructor (without parameters) for CPlugIn you must specify them at the constructor of your class:

```

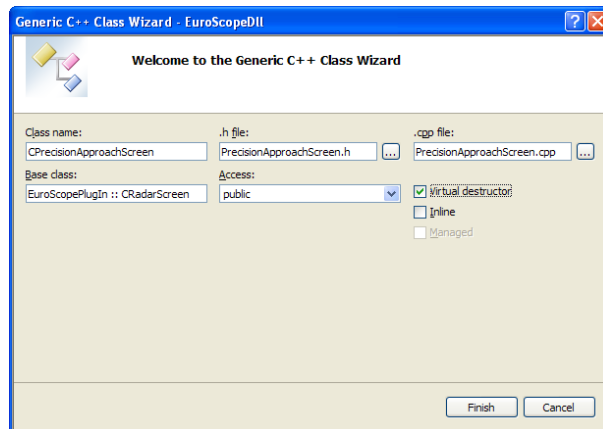
CPrecisionApproachPlugIn::CPrecisionApproachPlugIn( void )
    : CPlugIn ( EuroScopePlugIn::COMPATIBILITY_CODE,
                "Precision Approach Radar",
                "1.0.0",
                "Gergely Csernak",
                "Free to be distributed as source code" )
{
    // add your initialization here
}

```

This is actually enough. The plug-in is ready, you can load it via the *Other Settings/Plug-ins ...* menu. But practically it will not do anything.

My own radar screen

In this example I will create a complete new radar display. So I create a new class inherited from **CRadarScreen**. Use the add class wizard again:



Inherit from **EuroScopePlugIn::CRadarScreen**.

Add some data (description, threshold coordinates, altitude, runway heading and glide slope angle). I would like to store them in the ASR file:

```

CString      m_Description ;
CString      m_Latitude, m_Longitude ;
int          m_Altitude ;
int          m_Heading ;
double       m_Slope ;

```

I also initialized these variables in the constructor to LHB 31R:

```
m_Description = "LHB 31R PAR" ;  
m_Latitude    = "N047.25.24.615" ;  
m_Longitude   = "E019.17.35.260" ;  
m_Altitude    = 550 ;  
m_Heading     = 310 ;  
m_Slope       = 2.5 ;
```

There are four virtual functions of the **CRadarScreen** that will be overridden in my example.

OnAsrContentLoaded

In this function I load my variables from the ASR file. If they exist there then the default values are change.

OnAsrContentToBeSaved

This function is called by EuroScope when an ASR file is to be closed. Here I save back all the values to the ASR file. The new values are tested against the original ones and the user will be prompted to save the changes or not. You can save your actual values without testing if the values are different or not, EuroScope will test it.

OnRefresh

This is the most important function that refreshes the content of the screen. As this is my own display type it really does not matter in which phase I draw the content. Just be sure to refresh it only one time.

Most of the code is not EuroScope specific here. The only interesting code is the aircraft loop:

```
EuroScopePlugIn :: CRadarTarget rt ;  
for ( rt = GetPlugIn () -> RadarTargetSelectFirst () ;  
      rt.IsValid () ;  
      rt = GetPlugIn () -> RadarTargetSelectNext ( rt ) )
```

OnCompileCommand

This function tests if the command is “.par”, and if so it opens a dialog box to make the setup. If the OK was pressed at the end then the new values are saved.

OnCompileCommand

It is a must-be-implemented function. If you have nothing to do here, then add as an inline to the header:

```
inline virtual void    OnAsrContentToBeClosed ( void )  
{  
    delete this ;  
} ;
```

Register and create the radar screen

We have the complete radar screen now. The only step is missing is to register it, and when a new screen is created then allocate a new instance of my radar screen. I did both in the **CPrecisionApproachPlugIn** class.

In the constructor I registered my own view type:

```
RegisterDisplayType (  
    "Precision Approach View",  
    false,           // no radar content here  
    false,           // no geo referencing  
    true,            // it can be saved to an ASR file  
    true ) ;        // can be created
```

This will register the “Precision Approach View” type radar screens. In the Sector file/New ... menu you will find a “New Precision Approach View” menu item too. The new radar screen does not need normal radar content, not geo referenced (latitude and longitude has no meaning there), can be created and saved.

When the user asks for a *New Precision Approach View* the **OnRadarScreenCreated** of my plug-in instance is called. There I simply allocate a new instance of my radar. This instance will be called to setup and display the content.

```
if ( strcmp ( sDisplayName, "Precision Approach View" ) )  
    return NULL ;  
else  
    return new CPrecisionApproachScreen ;
```

Another example plug-in

The following codes are from the holding list plug-in. Here I will not describe all the lines, just the interesting ones that are missing from the previous.

My TAG items and item functions

Some constant definitions. The item codes:

```
const int TAG_ITEM_HOLDING_NAME = 1 ;  
const int TAG_ITEM_HOLDING_TIME = 2 ;  
const int TAG_ITEM_REMAINING_TIME = 3 ;
```

The functions (the first two are for the item functions, the last three is for the popup things):

```
const int TAG_FUNC_HOLDING_EDIT = 1 ;  
const int TAG_FUNC_HOLDING_WAIT = 2 ;  
  
const int TAG_FUNC_HOLDING_EDITOR = 10 ;  
const int TAG_FUNC_HOLDING_WAIT_LIST = 11 ;  
const int TAG_FUNC_HOLDING_WAIT_CLEAR = 12 ;
```

Then register my TAG items:


```

RegisterTagItemType ( "Holding point name",
                      TAG_ITEM_HOLDING_NAME ) ;

RegisterTagItemType ( "Holding time",
                      TAG_ITEM_HOLDING_TIME ) ;

RegisterTagItemType ( "Remaining holding time",
                      TAG_ITEM_REMAINING_TIME ) ;

```

And also my TAG functions:

```

RegisterTagItemFunction ( "Holding name editor",
                          TAG_FUNC_HOLDING_EDIT ) ;

RegisterTagItemFunction ( "Holding time popup",
                          TAG_FUNC_HOLDING_WAIT ) ;

```

And finally I need my own AC list. I have to save the reference to the list:

```

m_HoldingList = RegisterAcList ( "Holding list" ) ;

```

The registration also loads the user saved setup from the settings file. If empty or if we want to override it, define my own columns:

```

m_HoldingList.AddColumnDefinition (
    "C/S",
    6,
    false,
    NULL, EuroScopePlugIn :: TAG_ITEM_TYPE_CALLSIGN,
    NULL, EuroScopePlugIn :: TAG_ITEM_FUNCTION_NO,
    NULL, EuroScopePlugIn :: TAG_ITEM_FUNCTION_NO ) ;

m_HoldingList.AddColumnDefinition (
    "REM",
    6,
    true,
    MY_PLUGIN_NAME, TAG_ITEM_REMAINING_TIME,
    MY_PLUGIN_NAME, TAG_FUNC_HOLDING_WAIT,
    NULL, EuroScopePlugIn :: TAG_ITEM_FUNCTION_NO ) ;

```

Providing the TAG items

Override the `OnGetTagItem` to provide data for the defined item:

```

case TAG_ITEM_HOLDING_NAME :
    strcpy ( sItemString,
            m_HoldingData [ idx ].m_PointName ) ;
    break ;

case TAG_ITEM_HOLDING_TIME :
    sec = (int) ( time ( NULL ) -
                  m_HoldingData [ idx ].m_Entered ) ;
    sprintf ( sItemString,
            "%02d:%02d",

```

```

        sec / 60, sec % 60 ) ;
break ;

```

And in the last change the color to EMG color if the time is over:

```

if ( m_HoldingData [ idx ].m_Leave )
{
    if ( m_HoldingData [ idx ].m_Leave >= time ( NULL ))
        sec = (int) ( m_HoldingData [ idx ].m_Leave -
            time ( NULL )) ;
    else
    {
        sec = (int) ( time ( NULL ) -
            m_HoldingData [ idx ].m_Leave ) ;
        * pColorCode =
            EuroScopePlugIn :: TAG_COLOR_EMERGENCY ;
    }
    sprintf ( sItemString,
        "%02d:%02d",
        sec / 60, sec % 60 ) ;
}

```

Handle function calls

The calls may come from TAG items. It is from a TAG or from an AC list element. When clicking on a holding name, then allow editing it. And when the editing is finished, then save the result:

```

case TAG_FUNC_HOLDING_EDIT : // TAG function
    OpenPopupEdit ( Area,
        TAG_FUNC_HOLDING_EDITOR,
        m_HoldingData [ idx ].m_PointName ) ;

    break ;

case TAG_FUNC_HOLDING_EDITOR : // when finished editing
    m_HoldingData [ idx ].m_PointName = sItemString ;
    m_HoldingData [ idx ].m_PointName.MakeUpper () ;
    m_HoldingData [ idx ].m_Hard      = true ;
    break ;

```

When clicking on the remaining time item, then popup a list with some minutes available and a fixed element at the bottom to clear the downcounter. When something is selected, then save the value:

```

case TAG_FUNC_HOLDING_WAIT : // TAG function
    OpenPopupList ( Area, "Wait", 1 ) ;

    for ( i = 90 ; i > 30 ; i -= 10 )
    {
        str.Format ( "%d", i ) ;
        AddPopupListElement (
            str,

```

```

        "",
        TAG_FUNC_HOLDING_WAIT_LIST ) ;
    }

...

for ( ; i > 0 ; i -- )
{
    str.Format ( "%d", i ) ;
    AddPopupListElement (
        str,
        "",
        TAG_FUNC_HOLDING_WAIT_LIST,
        i == 10 ) ;
}

AddPopupListElement (
    "Clear",
    "",
    TAG_FUNC_HOLDING_WAIT_CLEAR,
    false,
    EuroScopePlugIn :: POPUP_ELEMENT_NO_CHECKBOX,
    m_HoldingData [ idx ].m_Leave == 0,
    true ) ;
break ;

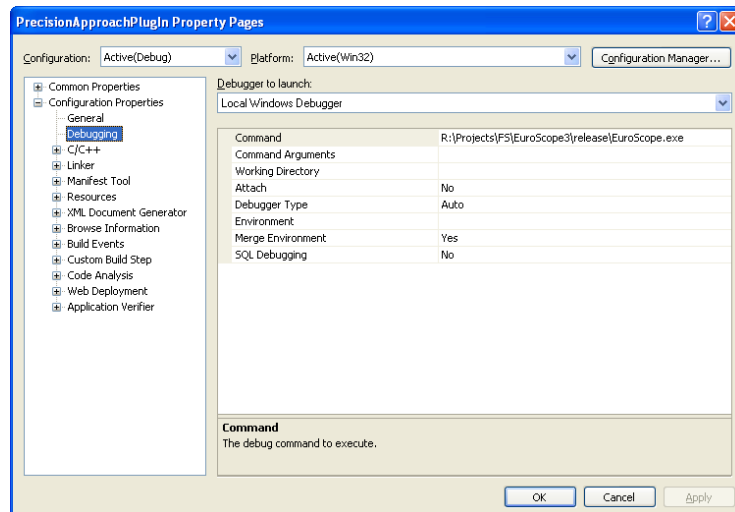
case TAG_FUNC_HOLDING_WAIT_LIST :
    // a value is selected from the popup list
    m_HoldingData [ idx ].m_Leave =
        time ( NULL ) + 60 * atoi ( sItemString ) ;
    break ;

case TAG_FUNC_HOLDING_WAIT_CLEAR :
    // clear the waiting time
    m_HoldingData [ idx ].m_Leave = 0 ;
    break ;

```

Debugging a plug-in

It is possible to debug a plug-in. In the project properties dialog specify EuroScope as the command to be executed:



Then go and load the debug version of your DLL. You will be able to define breakpoints inside your DLL and debug as normal.

Important: Never debug your plug-in when connected to VATSIM in an online session. The plug-in runs in the same thread as the rest of EuroScope. When you stop in a breakpoint the whole process is stopped. That may causes too big delay in processing the network protocol messages. That may cause problems in the server connection, that is strictly forbidden in VATSIM. Please, make a log from your session and use a playback session when debugging. That is also better to be able to reproduce your test cases.

Happy plug-in development.

Gergely Csernák

EuroScope developer